Business Requirements Document (BRD)

Overview

This document outlines the requirements for a new functionality in the API Encryption project hosted on GitHub. The goal is to enable users to view all decrypted data and provide an option to download this data in a tabular format. This enhancement aims to improve user experience by allowing easy access to decrypted information.

Existing System Summary

The current system comprises the following API endpoints:

1. **GET /test**
- **Controller**: MainController
- **Method**: testModel
- **Summary**: Returns a simple confirmation that the API is working.

2. **GET /enc**
- **Controller**: MainController
- **Method**: encryptModel
- **Summary**: Accepts data and a secret key to return the encrypted token.

3. **GET /dec**
- **Controller**: MainController
- **Method**: decryptModel
- **Summary**: Accepts an encrypted token and a secret key to return the decrypted value.

Proposed Changes

New Functionality

- **Endpoint**: `GET /decrypted-data`
- **Description**: This endpoint will return all decrypted data in a structured format.
- **Response Format**: JSON containing an array of decrypted records.
- **Download Option**:
- Implement a feature that allows users to download the decrypted data in a tabular format (CSV or Excel).

Sample Code Snippet

```java
@GetMapping("/decrypted-data")
public ResponseEntity<List<DecryptedData>> getDecryptedData() {
List<DecryptedData> decryptedDataList = fetchDecryptedData(); // Method to fetch decrypted data
```

```
return ResponseEntity.ok(decryptedDataList);

}

@GetMapping("/download-decrypted-data")

public void downloadDecryptedData(HttpServletResponse response) throws IOException {

List<DecryptedData> decryptedDataList = fetchDecryptedData(); // Method to fetch decrypted data

response.setContentType("text/csv");

response.setHeader("Content-Disposition", "attachment; filename=\"decrypted_data.csv\"");

PrintWriter writer = response.getWriter();

writer.println("Column1,Column2,Column3"); // Header

for (DecryptedData data : decryptedDataList) {

writer.println(data.getColumn1() + "," + data.getColumn2() + "," + data.getColumn3());

}

writer.flush();

}
```

Business Rules

1. Users must have the necessary permissions to access decrypted data.

2. The system should validate the secret key before returning decrypted data.

3. The downloaded file must be formatted correctly to ensure compatibility with spreadsheet applications.

APIs to Consider

- **GET /decrypted-data**: To retrieve decrypted data.
- **GET /download-decrypted-data**: To facilitate downloading of decrypted data in CSV format.

Risks & Assumptions

Risks

- **Data Security**: Exposing decrypted data may lead to security vulnerabilities if not properly managed.
- **Performance**: Fetching large datasets may impact system performance.

Assumptions

- Users have the necessary access rights to view and download decrypted data.
- The existing encryption/decryption mechanisms are robust and secure.

Next Steps

1. Review and approve the proposed changes with stakeholders.
2. Implement the new API endpoints and download functionality.
3. Conduct testing to ensure data security and performance.
4. Update documentation to reflect the new features and usage instructions.
5. Deploy changes to the production environment and monitor for any issues.